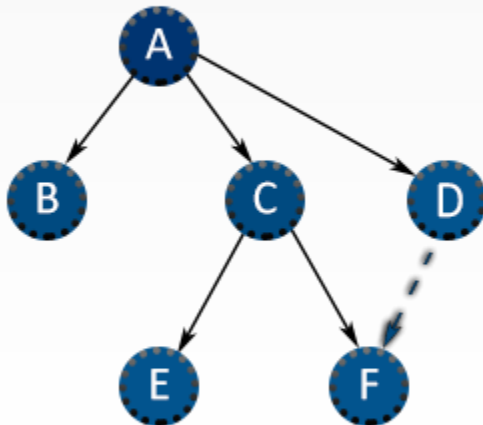


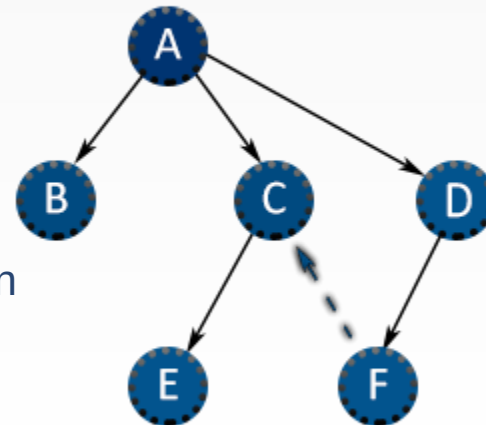
Grafos-2. Algoritmos de grafos

Discreta-2

BFS



DFS



Georgy Nuzhdin
2022-2023

Primeros algoritmos

- Grafo aleatorio
- Conversión de representación:
 - Lista de aristas a Matriz de Adyacencia
 - Matriz de Adyacencia a Matriz de Incidencia
 - Lista de aristas a Matriz de Incidencia
- Árbol aleatorio
- DFS
- BFS
- Comprobar que el grafo es conexo
- Contar las componentes conexas
- Búsqueda de radio y diámetro del grafo, excentricidad del vértice, distancia entre dos vértices
- (!) Generación de todos los árboles no isomorfos de N vértices (+1 punto por representación)
- (!) Búsqueda de la probabilidad de corte para un grafo aleatorio

El árbol recubridor

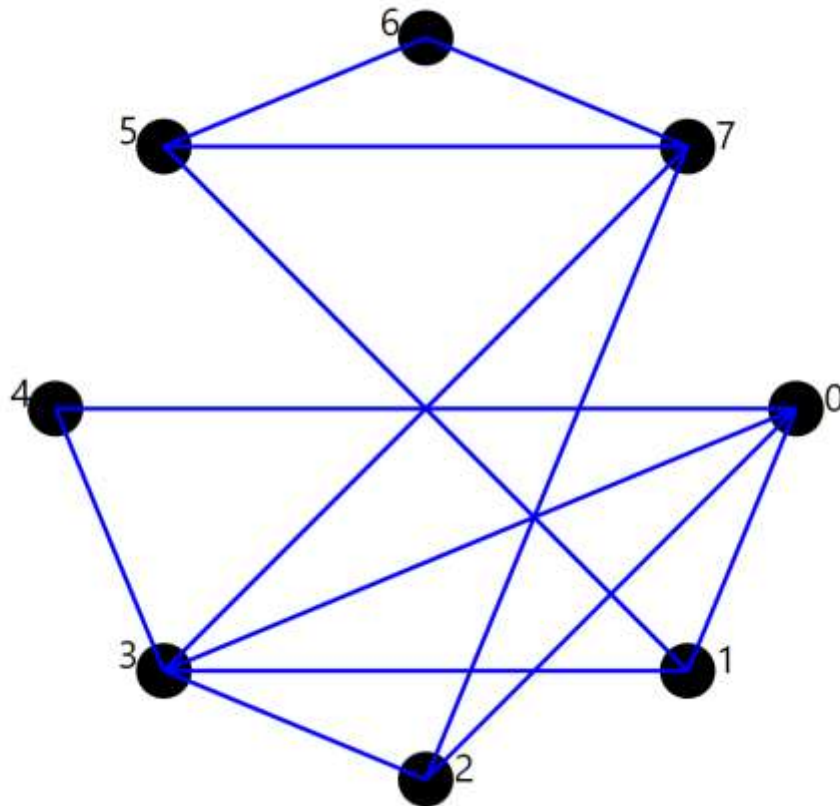
| Definición. El **árbol recubridor** del grafo G es aquel que contiene todos sus vértices

- Evidentemente, para cada grafo hay múltiples árboles recubridores

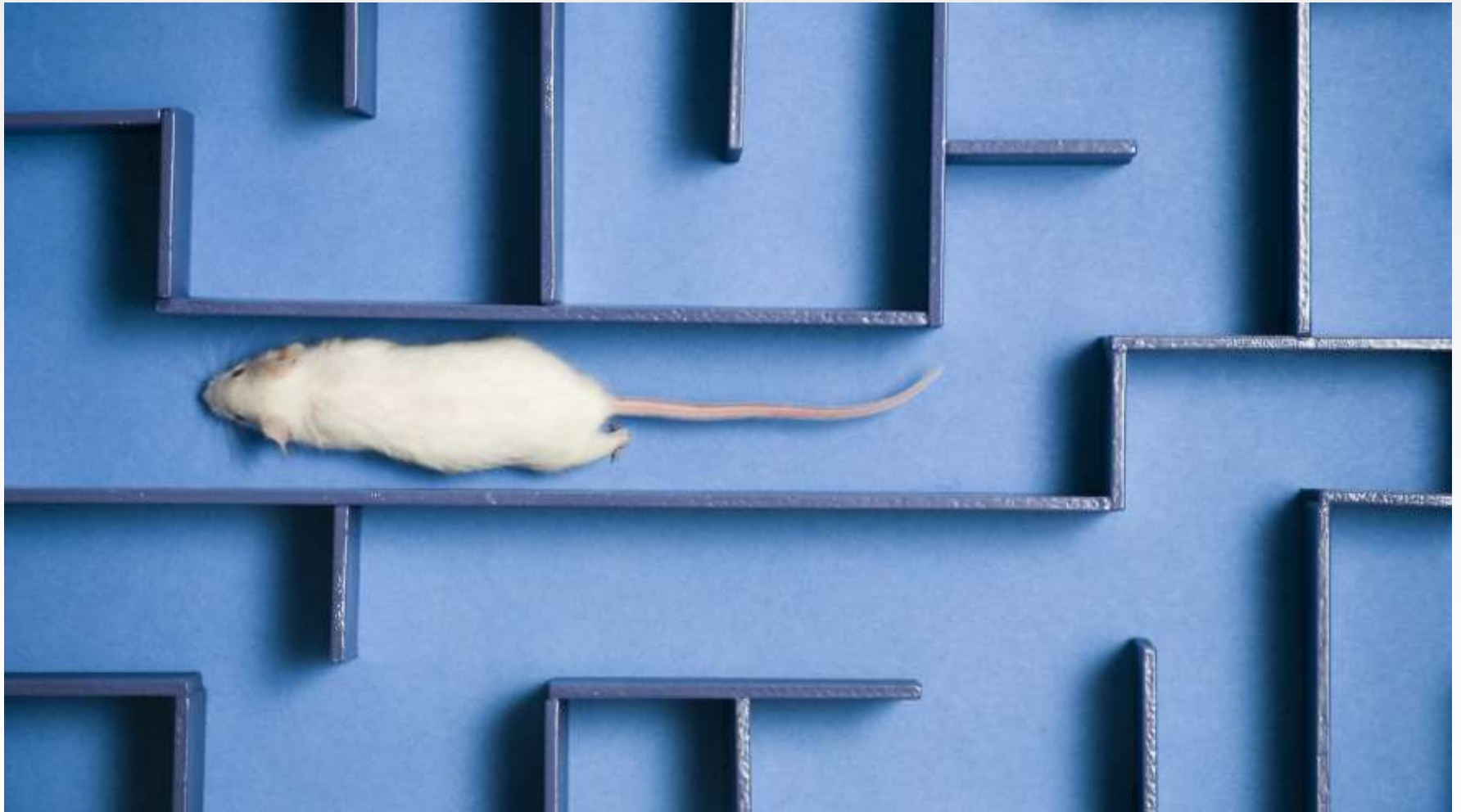
Depth First Search (Búsqueda en profundidad)

1. Numeramos los vértices si no están numerados
2. Creamos la lista vacía de aristas del árbol recubridor, la lista vacía de vértices visitados
3. Elegimos un vértice v_0 al azar (normalmente, el 0)
4. Entre sus adyacentes elegimos el de menor número v_i no visitado y nos desplazamos allí (añadimos el vértice v_i a la lista de vértices visitados y la arista v_0v_i a la lista de aristas del árbol recubridor)
Si no hay adyacentes no visitados, volvemos al vértice anterior
Si hemos vuelto al v_0 , no tiene adyacentes no visitados y no hemos recorrido todos los vértices, el grafo NO es conexo
5. Si no hemos recorrido todos los vértices, volvemos al punto 4

Generar un árbol recubridor usando DFS



Búsqueda en un laberinto



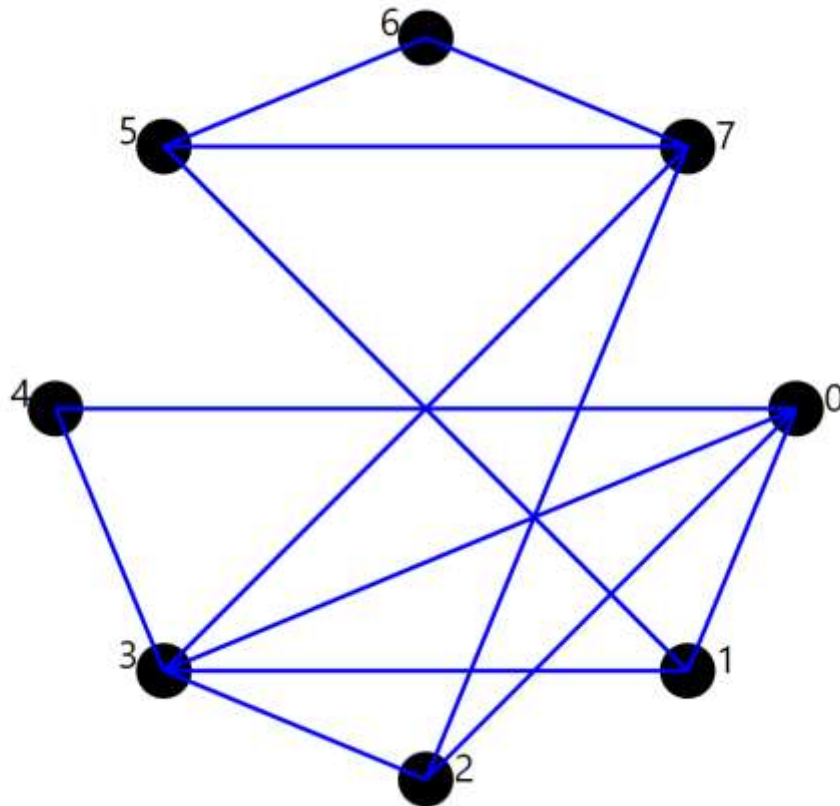
Depth First Search

- Crea distintos árboles recubridores en función del vértice de partida y la numeración
- Busca “la autopista más larga” posible
- Es cómodo cuando se trata de crear un camino con el mínimo de ramificaciones (por ejemplo, para construir vías de tren o tranvía)

Breadth First Search (Búsqueda en anchura)

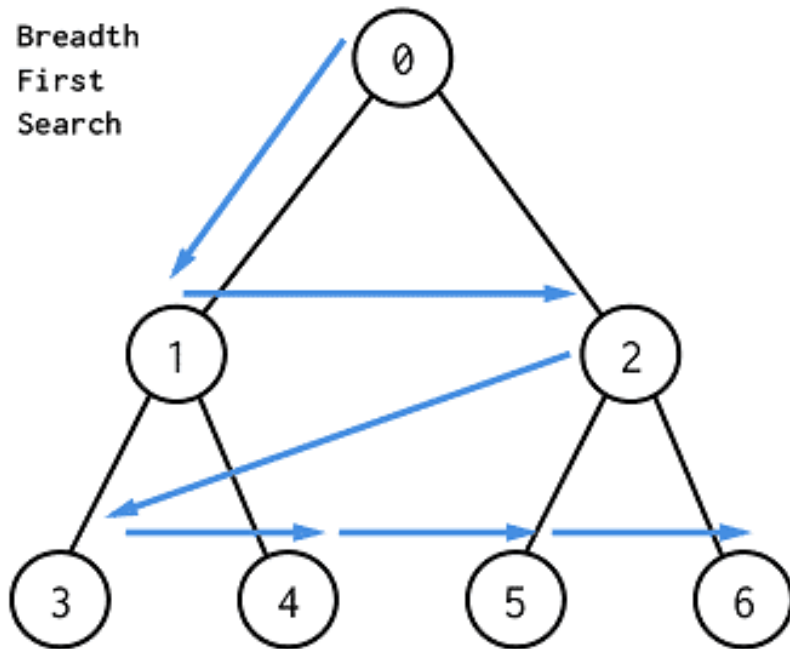
1. Numeramos los vértices si no están numerados
2. Creamos la lista vacía de aristas del árbol recubridor, la lista vacía de vértices visitados
3. Elegimos un vértice v_0 al azar (normalmente, el 0)
4. Añadimos TODOS los vértices adyacentes v_i a la lista de vértices visitados y las arista v_0v_i a la lista de aristas del árbol recubridor
5. Para cada v_i repetimos el paso 4
Si no quedan adyacentes no visitados y no hemos recorrido todos los vértices, el grafo NO es conexo

Generar un árbol recubridor usando BFS

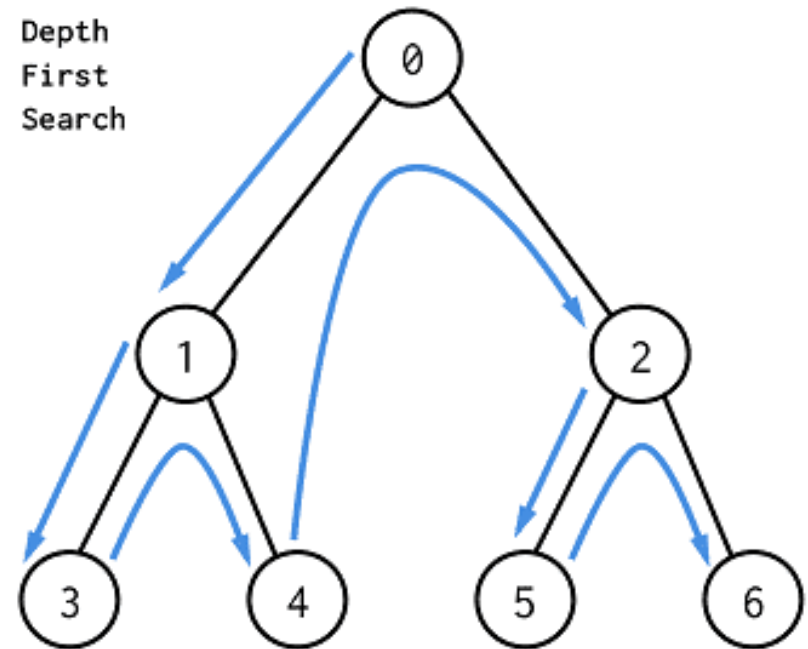


BFS vs DFS

Breadth
First
Search



Depth
First
Search



Puentes y vértices de corte

| Definición. Un **punto** es una arista tal que al eliminarla, aumenta la cantidad de componentes conexas

| Definición. Un **vértice de corte** es un vértice tal que al eliminarlo, aumenta la cantidad de componentes conexas

- Los puentes y vértices de corte son aristas y vértices que efectúan la conexión entre distintas zonas del grafo

Distancias en el grafo

- | La **distancia** entre dos vértices es la longitud del recorrido más corto entre ellos (la cantidad mínima de aristas que los unen)
- | La **excentricidad** de un vértice es la mayor de las distancias a otros vértices
- | El **radio** del grafo es la menor de las excentricidades de sus vértices, mientras que el **diámetro** es la mayor

¿Qué búsqueda (DFS o BFS) es mejor para

- Calcular la distancia entre dos vértices?
- Hallar el radio de un grafo?
- Buscar puentes?
- Buscar componentes conexas?

Búsqueda de componentes conexas

- Realizamos la búsqueda en profundidad (DFS) partiendo de cualquier vértice
- Cuando termine, partimos de cualquier vértice no visitado
- Complejidad = $O(|A| + |V|)$

Probabilidad de ser conexo (probabilidad de corte)

- Para N vértices determinar p para que el grafo aleatorio (p, N) sea conexo con la probabilidad
 - Del 90%
 - Del 50%
- Otro punto más: determinar la función $f(N) = p \mid (p, N) \text{ conexo con la probabilidad del 50\%}$

Búsqueda de puentes

- Idea “naïf”: si quitamos la arista, el grafo deja de ser conexo
- ¿Por qué es mala?
- ¡Por la complejidad computacional! Será proporcional al cuadrado del tamaño del grafo
- ¡Usemos DFS!
- Está claro que las aristas que NO están en el árbol DFS no pueden ser puentes (forman ciclos)
- Pero NO TODAS las aristas del árbol DFS son puentes

Búsqueda de puentes

- IDEA: estamos analizando una arista XY del árbol DFS. Si ningún padre de X está conectado a ningún hijo de Y , es un puente
- Sea $t(v)$ la hora de entrada en el vértice v .
- Sea $\min(v)$, el mínimo entre $t(v)$, $t(v_{antes})$ para todos los vértices anteriores y $\min(v_{post})$ para todos los vértices posteriores en el árbol
- Nuestra arista $(v - x)$ es un puente si y sólo si $\min(x) > t(v)$
- `void dfs(int v, int p = -1) {`
 - `visited[v] = true;`
 - `tin[v] = low[v] = timer++;`
 - `for (int to : adj[v]) {`
 - `if (to == p) continue;`
 - `if (visited[to]) { low[v] = min(low[v], tin[to]); }`
 - `else {`
 - `dfs(to, v); low[v] = min(low[v], low[to]);`
 - `if (low[to] > tin[v]) IS_BRIDGE(v, to); }`
 - `}`
- `}`

Breadth First Search

- Crea distintos árboles recubridores en función del vértice de partida y la numeración
- Nos permite calcular la **distancia** entre dos vértices, la excentricidad, el radio y el diámetro del grafo
- Es cómodo cuando se trata de buscar el camino más corto entre dos vértices

Algoritmos de distancias

- Para calcular la distancia entre dos vértices, realizamos BFS desde uno de ellos y calculamos el nivel del otro en el árbol resultante
- Para calcular el diámetro y el radio, realizamos BFS desde todos los vértices
- Para calcular la excentricidad, realizamos BFS desde el vértice en cuestión y buscamos la rama más larga

El árbol de intervalos

- Tenemos N números
- Queremos saber sumarlos en $O(\log N)$

CONSTRUCCIÓN DE ÁRBOLES RECUBRIDORES MÍNIMOS

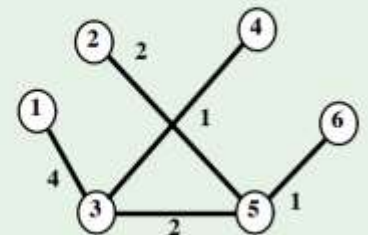
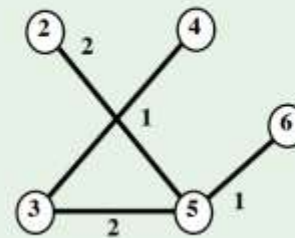
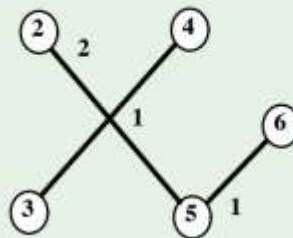
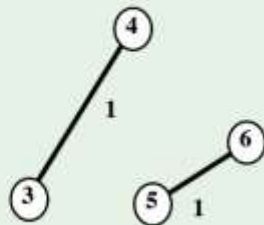
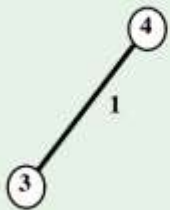
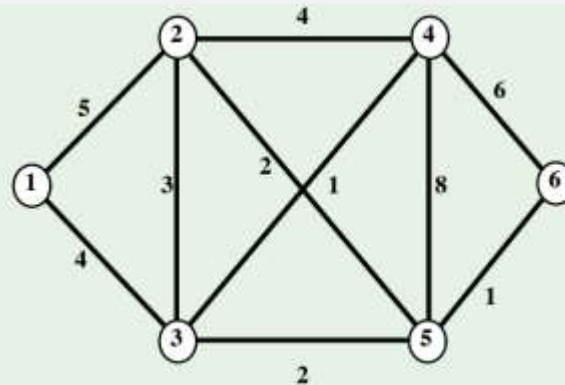
Tarea:

- En un grafo ponderado encontrar un árbol recubridor de peso mínimo (que la suma del peso de todas las aristas sea mínima)

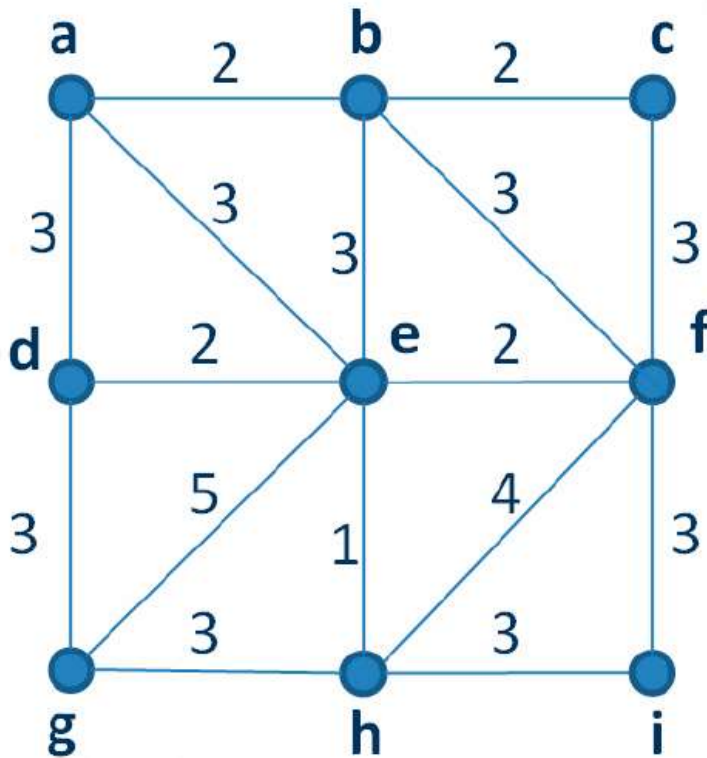
Algoritmo de Kruskal

1. Se parte del árbol vacío
2. Se le añade la arista de menor peso
3. Se van eligiendo aristas de menor peso entre las restantes
4. Para cada arista nueva se comprueba si al añadirla al árbol no se forman ciclos. En este caso, se añade
5. Volvemos al 3

Algoritmo de Kruskal en acción



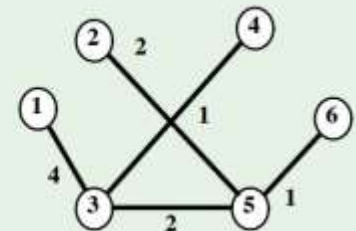
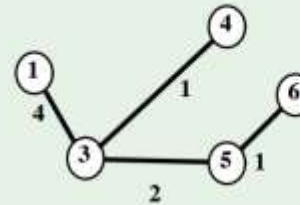
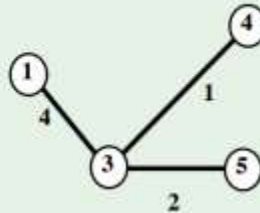
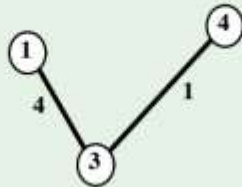
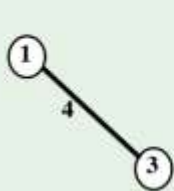
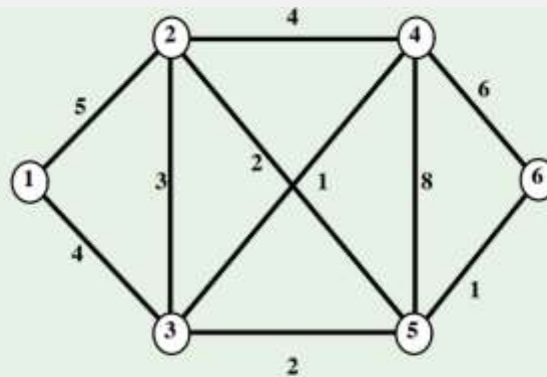
Busca el árbol recubridor mínimo (alg. Kruskal)



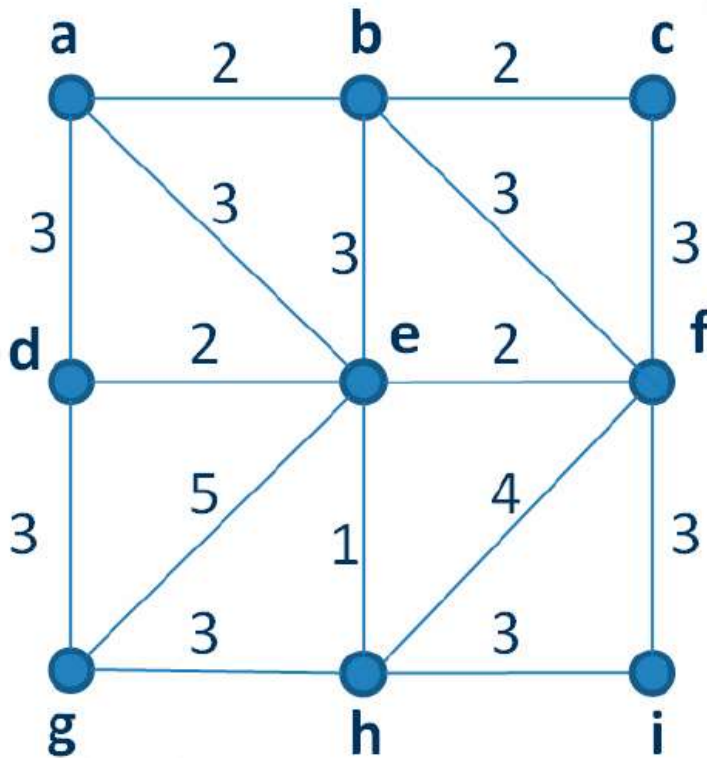
Algoritmo de Prim

1. Partimos de un vértice arbitrario
2. Entre las aristas adyacentes elegimos la que menor peso tiene. Será la primera arista del árbol que estamos construyendo
3. Entre los vértices adyacentes al árbol hallamos la arista que menor peso tiene. Si no forma ciclos, la añadimos al árbol
4. Volvemos al paso 3

Algoritmo de Prim en acción



Busca el árbol recubridor mínimo (alg. Prim)



Búsqueda de caminos mínimos (Edsger Dijkstra, 1959)

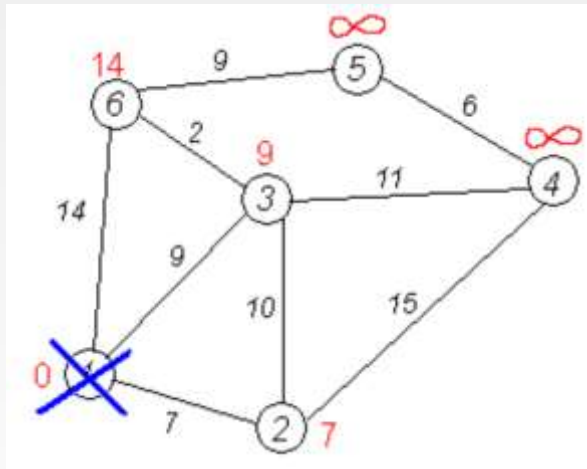
- *Permite encontrar el camino más corto de un vértice a todos los demás*
- $O(|V|^2 + |A|)$



Búsqueda de caminos mínimos (Dijkstra)

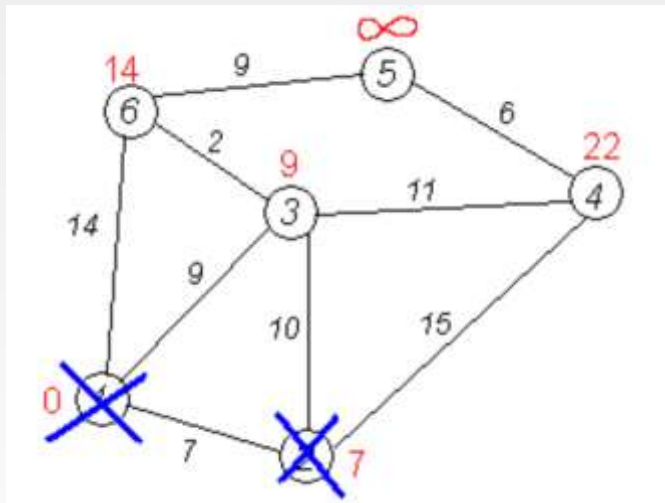
1. Fijamos la distancia a todos los vértices a infinito
2. Fijamos el vértice de partida y distancia hasta él = 0. Lo marcamos como visitado
3. Creamos una tabla de $N \times 2$. En la primera fila colocamos todos los vértices. En la segunda escribimos la distancia a todos los vértices (al principio, la distancia al vértice de partida es 0 y a las demás, infinito)
4. Buscamos el vértice de menor distancia M . Lo marcamos como visitado. Miramos todos los vértices adyacentes no visitados. Si la distancia al vértice de partida pasando por M disminuye, la cambiamos y (¡importante!) en la tabla de vértices añadimos una nueva fila con las distancias recalculadas y señalando en negrita el vértice M
5. Si no hemos visitado todos los vértices, volvemos al paso 4

Búsqueda de caminos mínimos (Dijkstra)



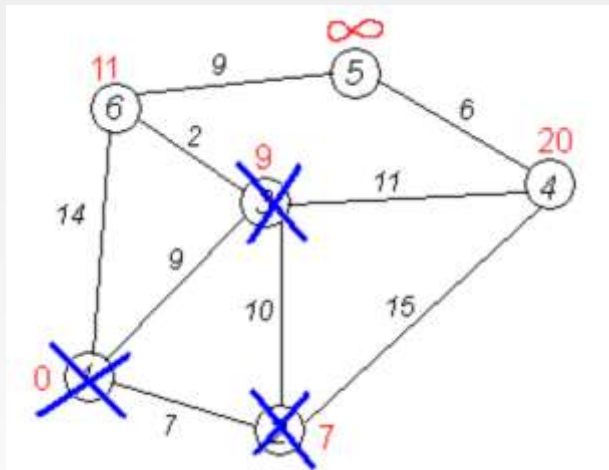
1	2	3	4	5	6
0	7	9	∞	∞	14

Búsqueda de caminos mínimos (Dijkstra)



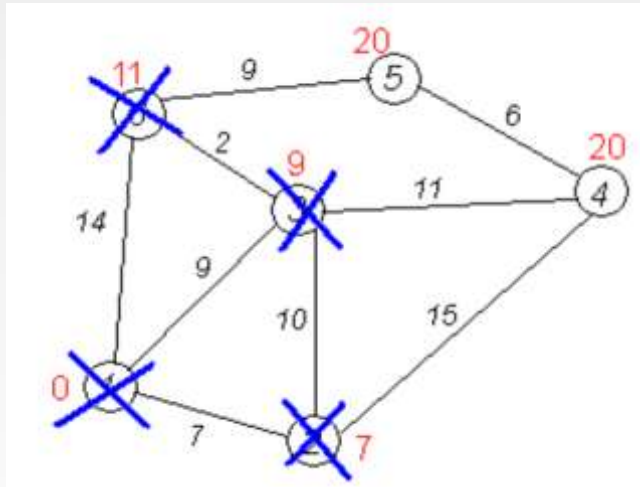
1	2	3	4	5	6
0	7	9	∞	∞	14
0	7	9	22	∞	14

Búsqueda de caminos mínimos (Dijkstra)



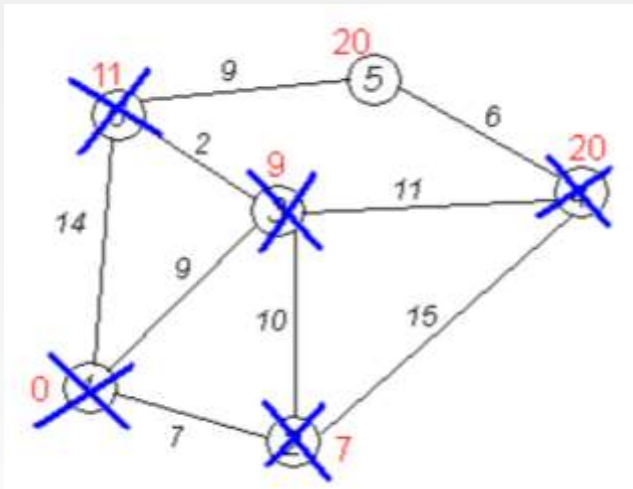
1	2	3	4	5	6
0	7	9	∞	∞	14
0	7	9	22	∞	14
0	7	9	20	∞	11

Búsqueda de caminos mínimos (Dijkstra)



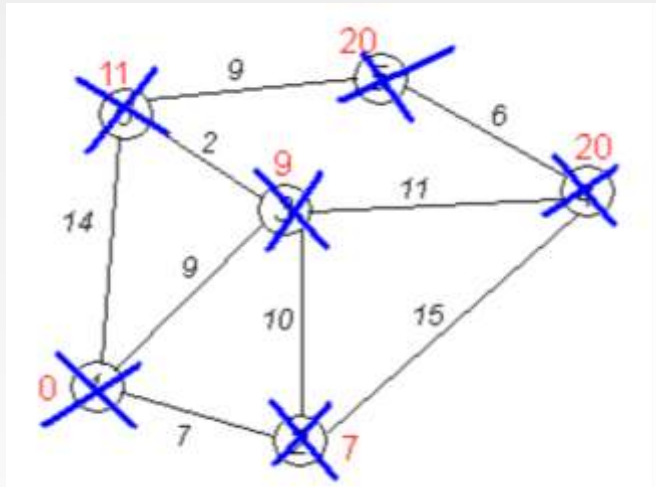
1	2	3	4	5	6
0	7	9	∞	∞	14
0	7	9	22	∞	14
0	7	9	20	∞	11
0	7	9	20	20	11

Búsqueda de caminos mínimos (Dijkstra)



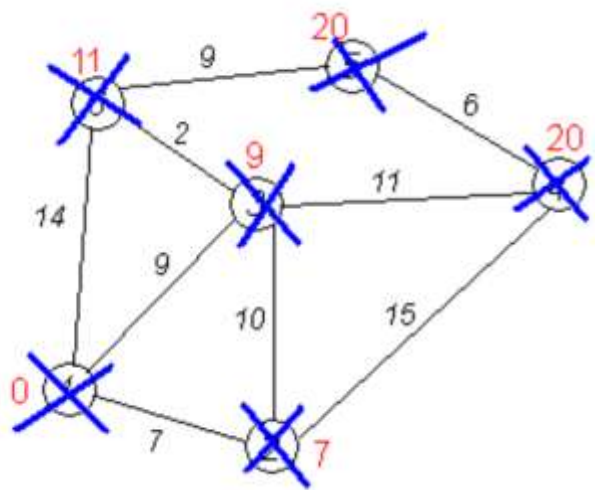
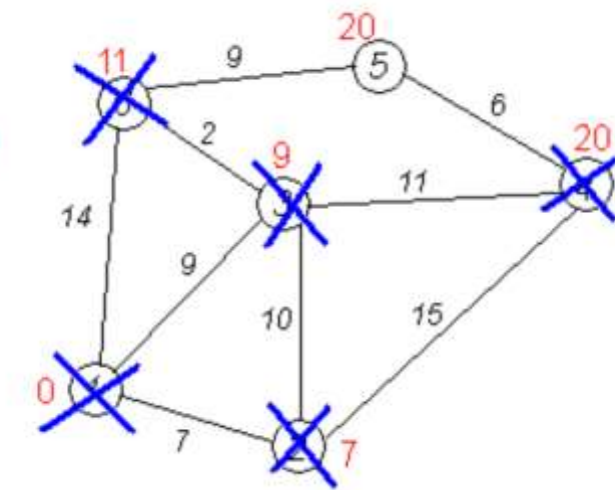
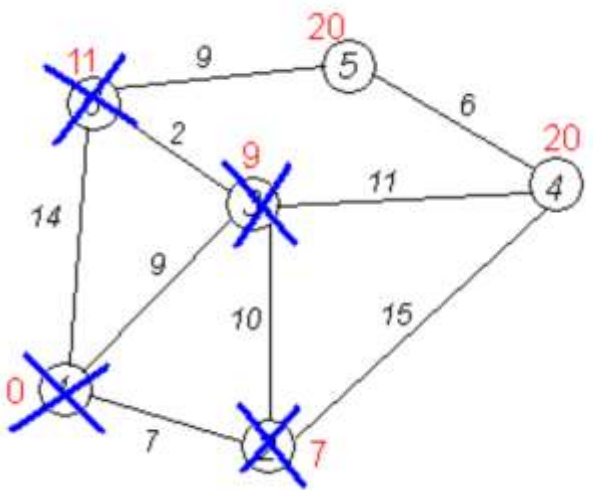
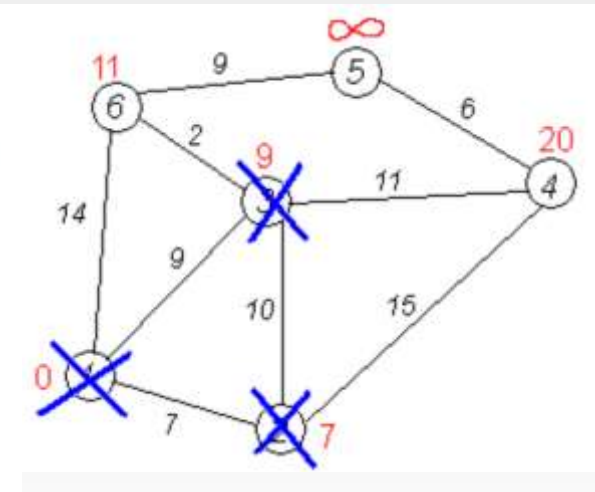
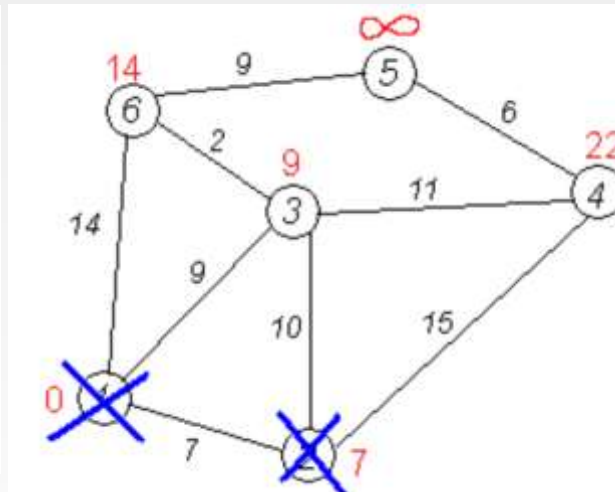
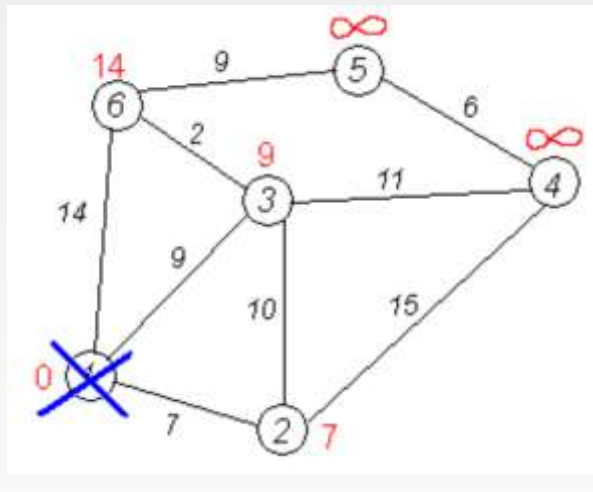
1	2	3	4	5	6
0	7	9	∞	∞	14
0	7	9	22	∞	14
0	7	9	20	∞	11
0	7	9	20	20	11
0	7	9	20	20	11

Búsqueda de caminos mínimos (Dijkstra)

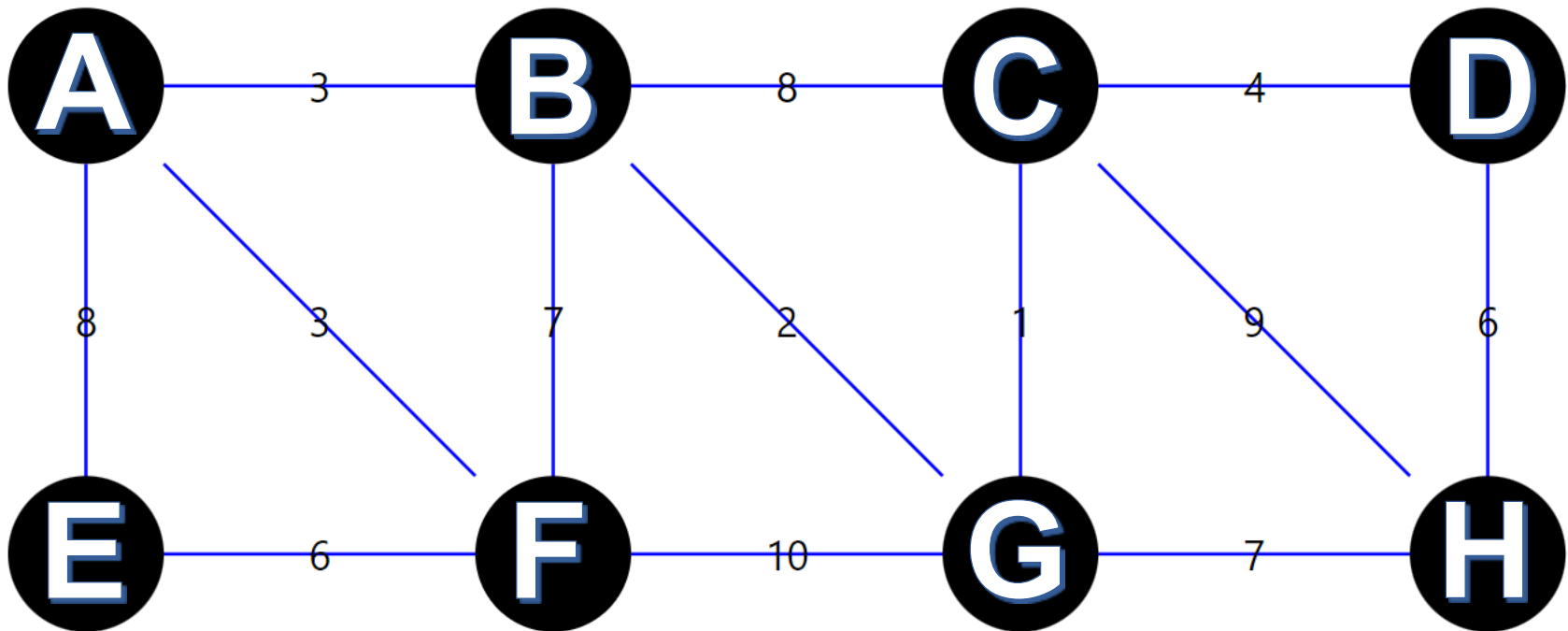


1	2	3	4	5	6
0	7	9	∞	∞	14
0	7	9	22	∞	14
0	7	9	20	∞	11
0	7	9	20	20	11
0	7	9	20	20	11
0	7	9	20	20	11

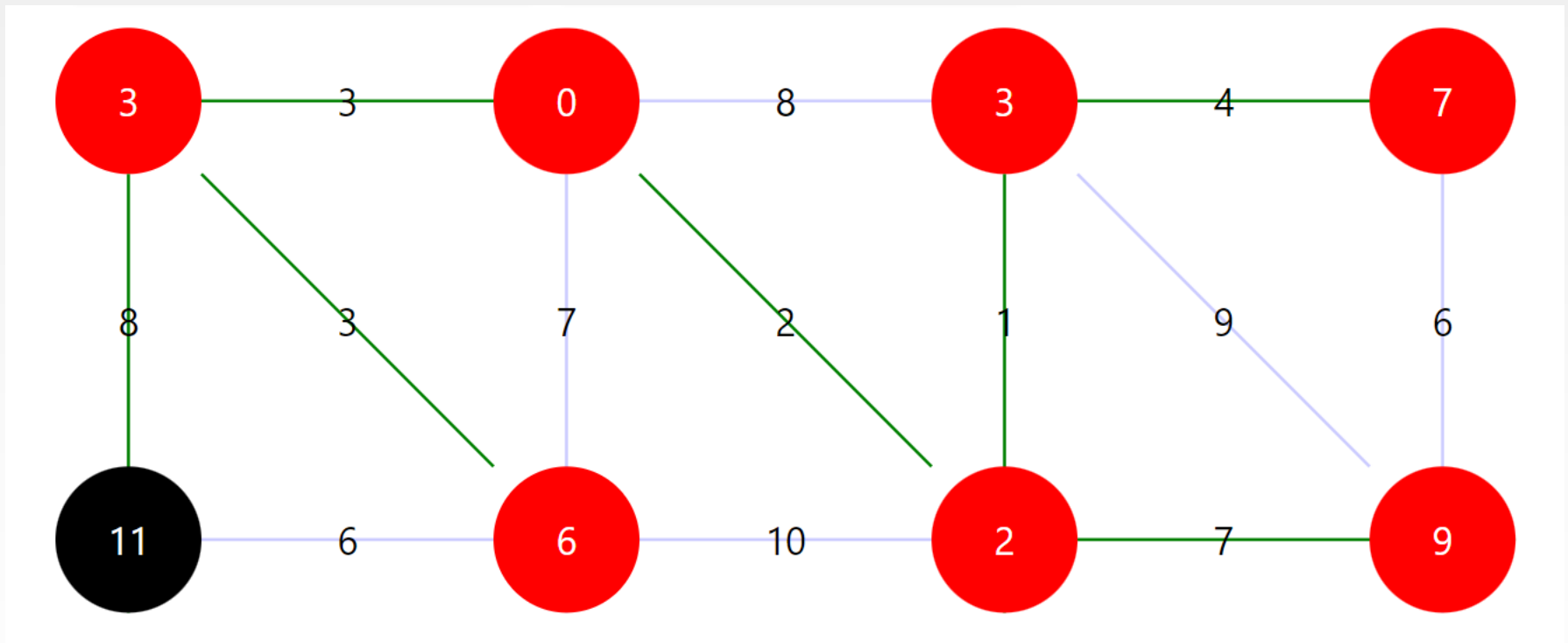
Búsqueda de caminos mínimos (Dijkstra)



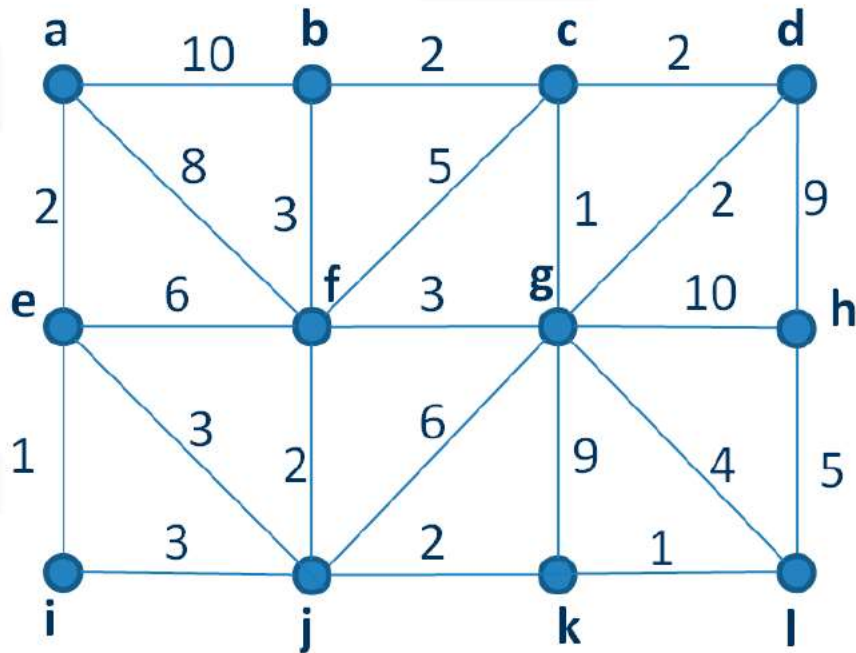
Construye el árbol de caminos mínimos a partir del B



Árbol de caminos mínimos



Busca el árbol de caminos mínimos



Búsqueda de flujo máximo

- Pensemos en un grafo ponderado como en una red de flujos, donde el peso de cada arista indica el máximo flujo posible
- Consideremos fenómenos con la ley de conservación de flujo:
el flujo saliente no puede superar el flujo entrante
- Ejemplos:
 - Tráfico (no pueden salir más coches de los que entran)
 - Corriente eléctrica (conservación de energía)
 - Flujo de gases o de líquidos por tuberías
 - Paquetes de información en internet
 - Transporte de mercancías
 - ...
- La tarea es determinar el flujo máximo posible entre dos vértices

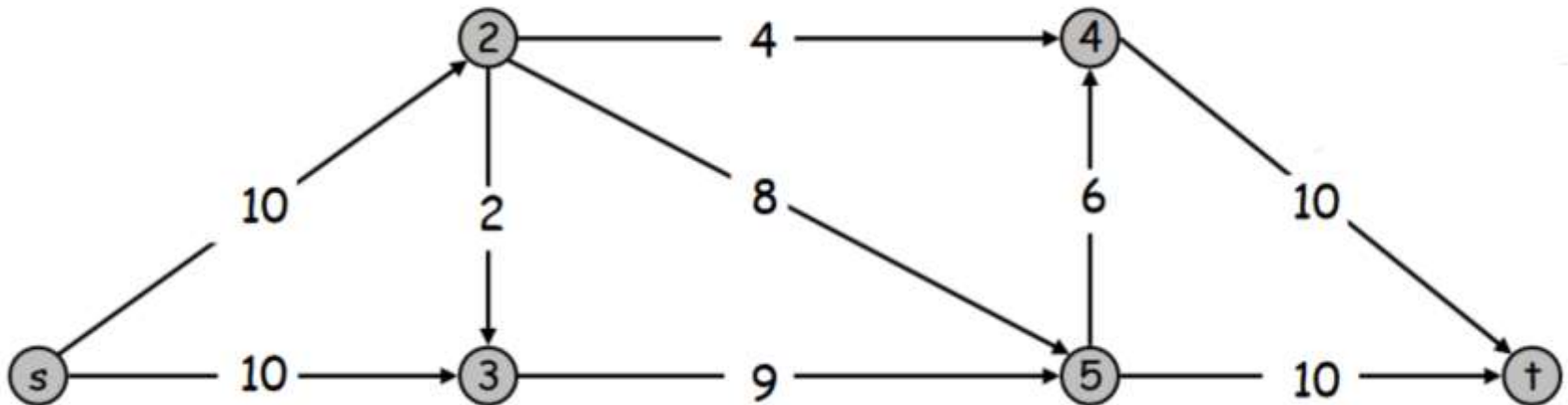
Teorema de corte mínimo (min-cut) (1956)



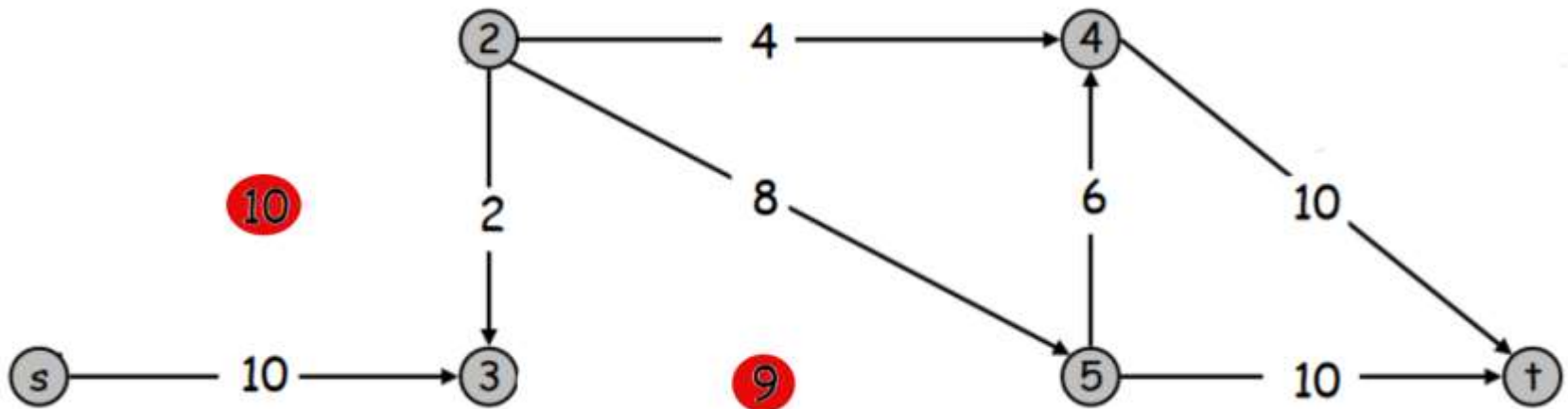
- Un **corte** consiste en la eliminación de ciertas aristas que desconectan el vértice origen del de destino
- El peso de un corte es la suma de pesos de las aristas eliminadas
- Teorema: **El flujo máximo es igual al peso del corte mínimo**



Busca el corte mínimo en este grafo



El corte mínimo en este grafo = 19



Algoritmo de Ford-Fulkerson (1956)

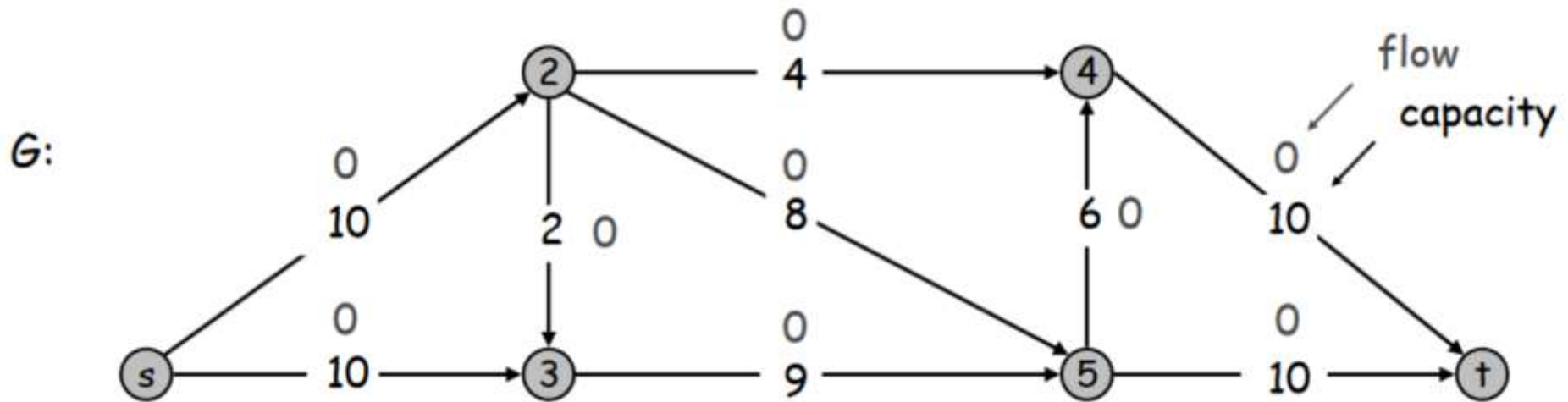


- Creamos una **copia del grafo** y fijamos todos los flujos a 0
- Buscamos un camino entre el origen y el destino en el
- Grafo original: Elegimos la arista de menor flujo. Restamos este valor a las demás aristas. A esta arista le damos la vuelta, a las demás dibujamos el flujo inverso (si potencialmente pueden ir en otra dirección)
- Copia del grafo: actualizamos los valores de flujo
- Repetimos el procedimiento hasta que no queden caminos entre el origen y el destino



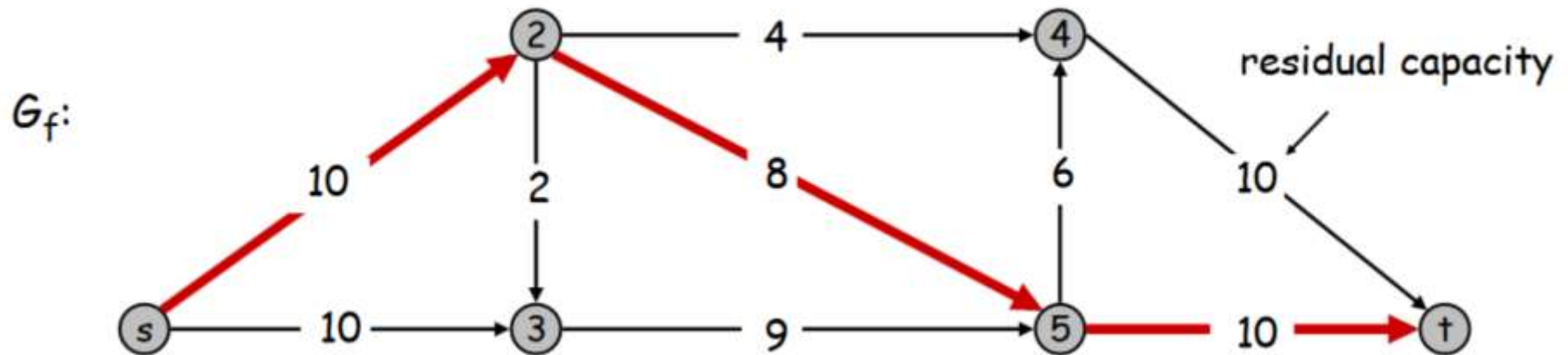
Algoritmo de Ford-Fulkerson (1)

- Creamos una copia del grafo inicial
- Fijamos el flujo de todas las aristas en 0



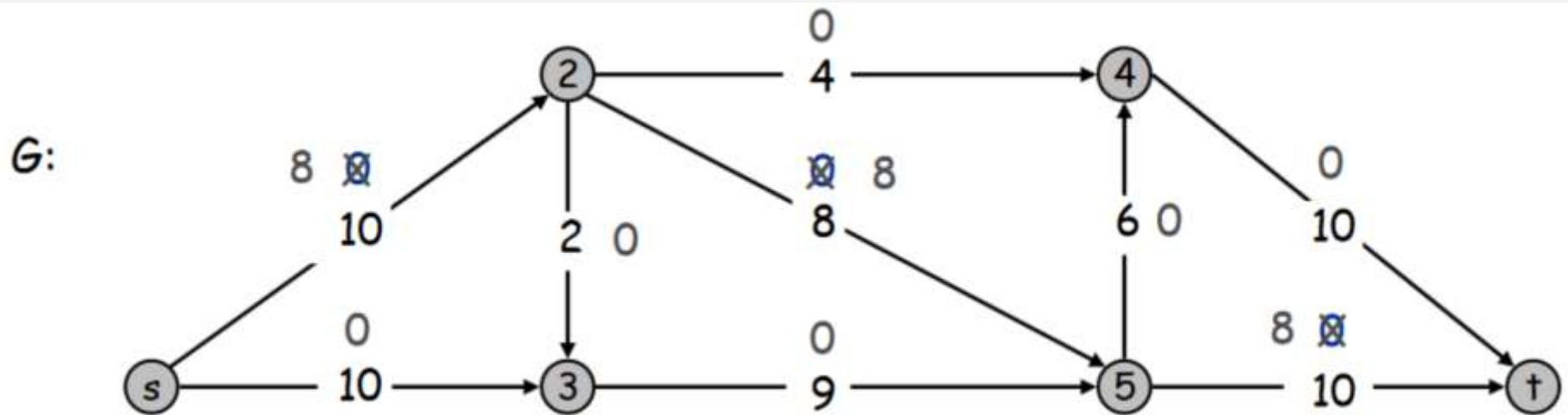
Algoritmo de Ford-Fulkerson (2)

- Elegimos un camino cualquiera entre el origen y el destino



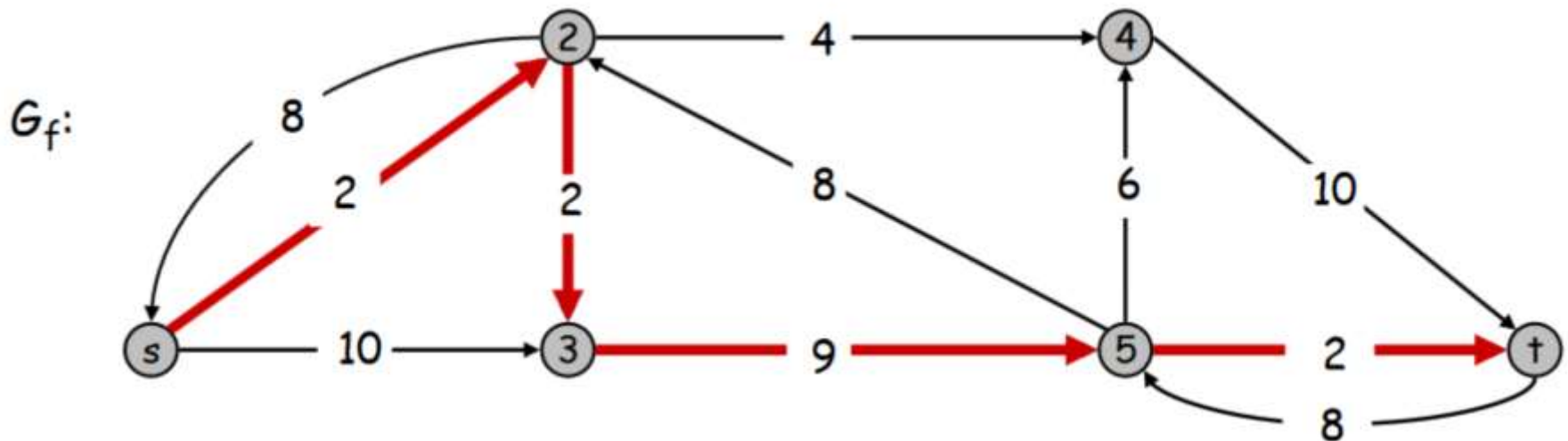
Algoritmo de Ford-Fulkerson (3)

- Actualizamos el flujo (ahora es 8 más que antes)



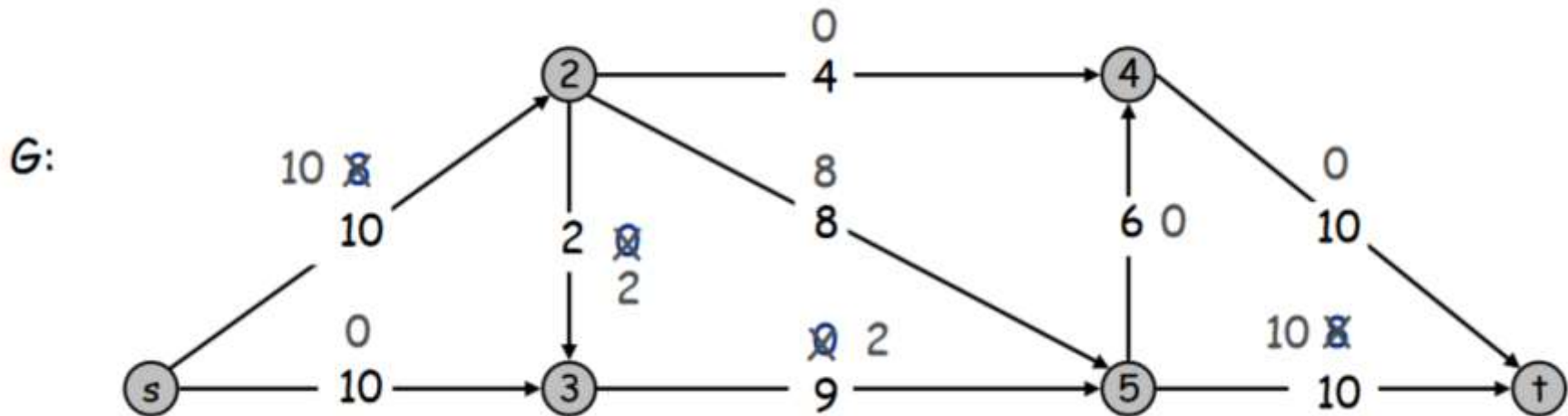
Algoritmo de Ford-Fulkerson (4)

- Dibujamos el flujo potencial inverso. La arista de flujo mínimo (la del medio, 8) la invertimos. A las demás les añadimos el flujo inverso
- Elegimos un nuevo camino entre el origen y el destino
- Su capacidad máxima es igual a 2
- Revertimos la flecha del 2, a las demás les sumamos 2 en dirección contraria

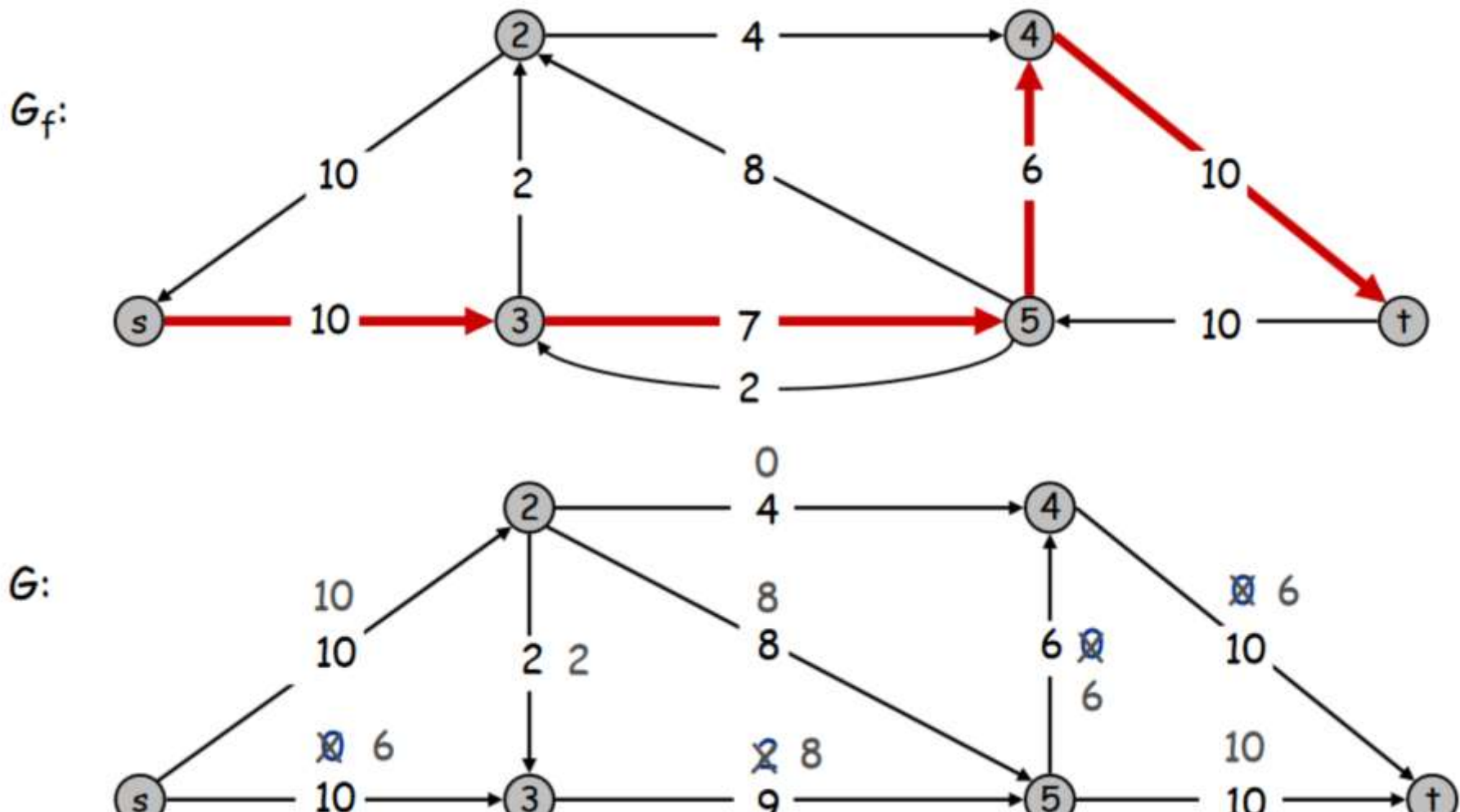


Algoritmo de Ford-Fulkerson (5)

- Actualizamos el flujo (ahora es 2 más que antes)

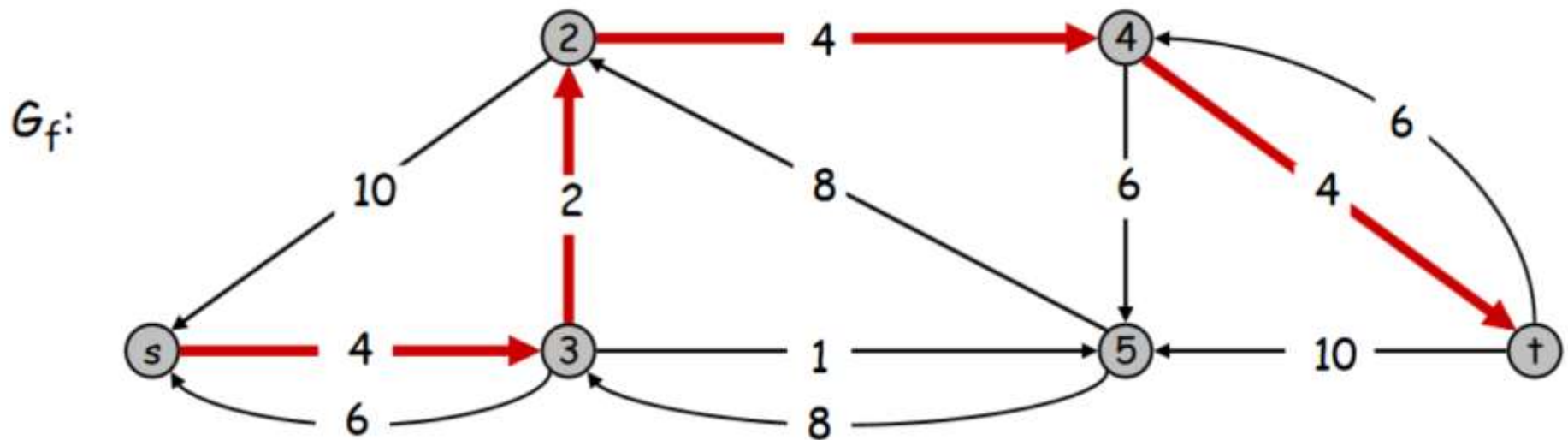


Un nuevo camino y su actualización



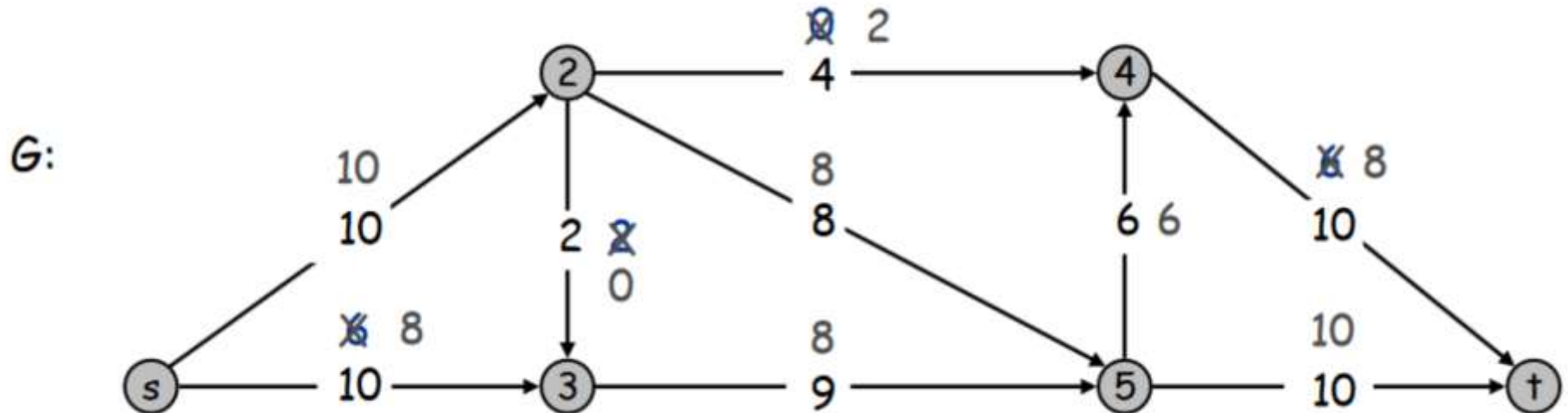
Algoritmo de Ford-Fulkerson (7)

- La flecha del 2 ahora va hacia arriba y la podemos aprovechar (recuerda que este flujo es potencialmente en ambas direcciones)

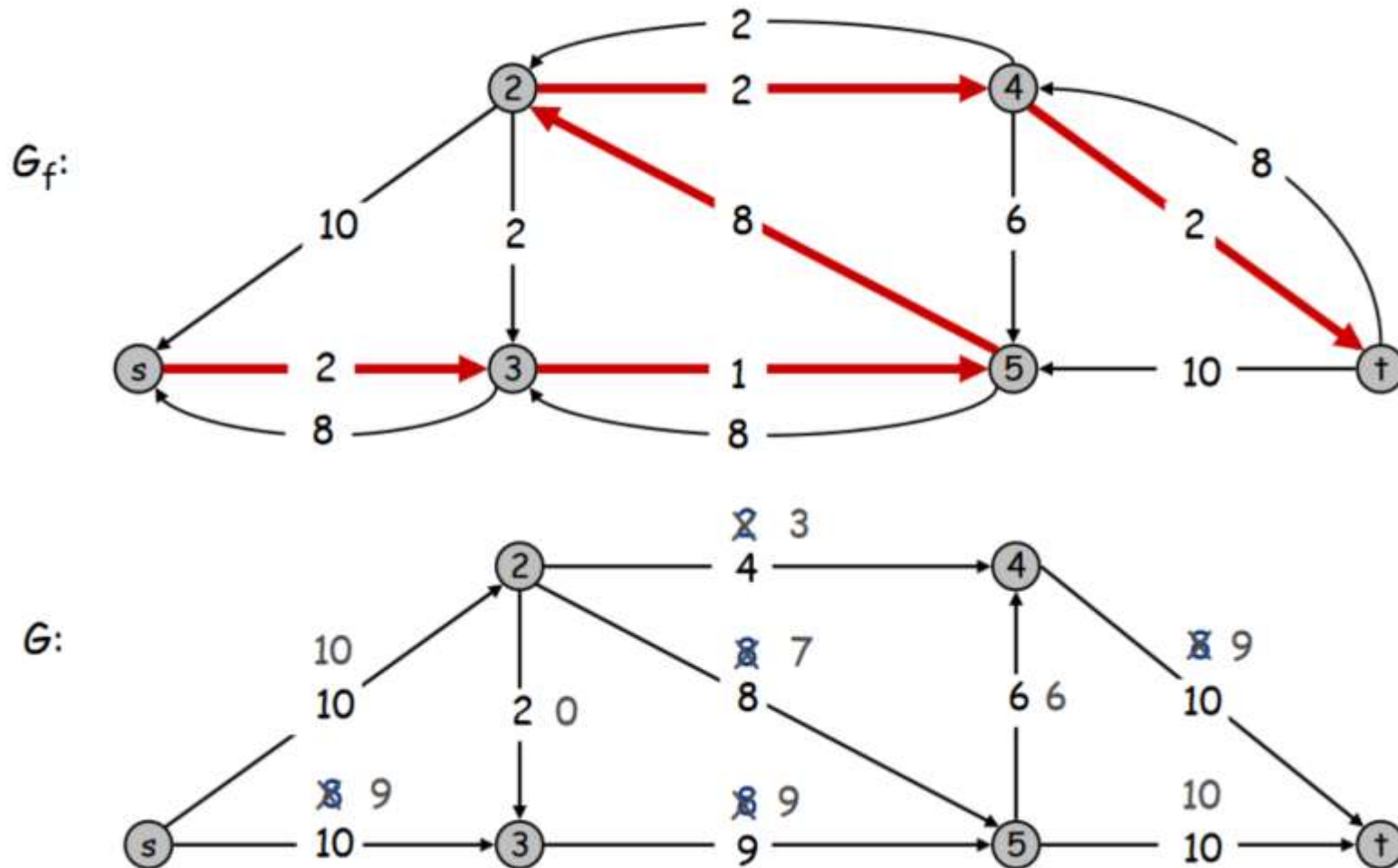


Algoritmo de Ford-Fulkerson (8)

- Fíjate en que hemos RESTADO el flujo a la flecha del 2 porque va en dirección contraria al grafo original

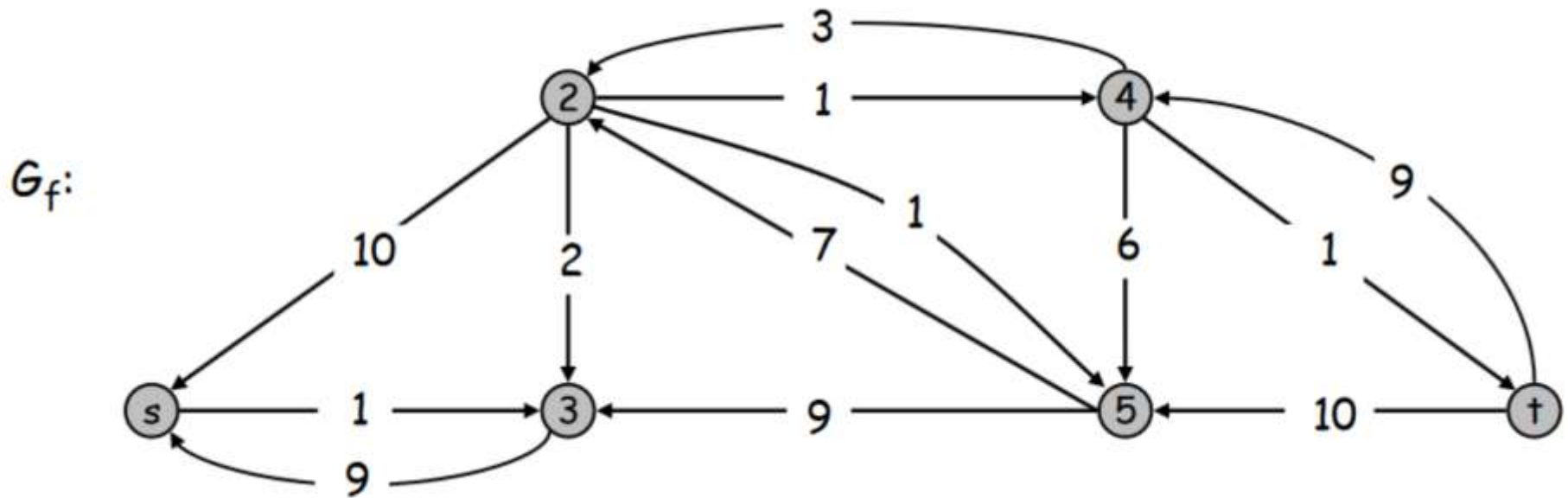


Queda el último camino (9)



Fin del algoritmo (10)

- No se puede llegar ya del vértice S al T



Busca el flujo máximo usando el Ford-Fulkerson

